

Video Node Installation Guide (Clean Raspberry Pi Setup)

This guide installs one camera node on a clean Raspberry Pi system.

The node:

- captures continuous rolling video
- stores short segments in RAM
- creates triggered clips
- uploads clips to the central server
- runs fully inside Docker Compose

Recommended hardware:

- Raspberry Pi 4 or 5
- Logitech C922 webcam
- Official Raspberry Pi PSU
- Raspberry Pi OS Lite 64-bit

System Overview

Node responsibilities:

```
graph TD; A[USB webcam] --> B[continuous FFmpeg recording]; B --> C[RAM segment ring buffer]; C --> D[trigger event]; D --> E[MP4 clip generation]; E --> F[upload to central server];
```

Recommended Hardware

Raspberry Pi

Recommended:

- Raspberry Pi 5 (preferred)
- Raspberry Pi 4 (works)

Memory:

- 4 GB minimum
 - 8 GB preferred
-

Webcam

Recommended:

- Logitech C922

Supported modes:

- MJPEG 1280x720 30fps
 - MJPEG 1920x1080 30fps
-

Power Supply

IMPORTANT:

Use official Raspberry Pi PSU.

Insufficient PSU causes:

- FFmpeg crashes
 - USB instability
 - camera disconnects
 - encoding failures
-

Install Raspberry Pi OS

Recommended:

- Raspberry Pi OS Lite 64-bit

Download:

- <https://www.raspberrypi.com/software/>
-

Initial System Setup

Update system:

```
sudo apt update
sudo apt upgrade -y
```

Install useful tools:

```
sudo apt install -y
git
curl
nano
htop
v4l-utils
```

Reboot:

```
sudo reboot
```

Configure WiFi

Edit:

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Add:

```
country=SE

network={
    ssid="archeryNet"
    psk="archery2026"
    priority=100
}
```

IMPORTANT:

Remove old WiFi networks if this node should only use archeryNet.

Reboot:

```
sudo reboot
```

Verify:

```
hostname -I
```

Expected:

```
192.168.60.xxx
```

Disable WiFi Power Saving

Temporary test:

```
sudo iw dev wlan0 set power_save off
```

Persistent configuration:

Create:

```
sudo mkdir -p /etc/NetworkManager/conf.d
```

```
sudo nano /etc/NetworkManager/conf.d/wifi-powersave.conf
```

Contents:

```
[connection]
wifi.powersave = 2
```

Restart:

```
sudo systemctl restart NetworkManager
```

Verify Webcam

Connect webcam.

Check device:

```
ls -al /dev/video0
```

Expected:

```
/dev/video0
```

List supported modes:

```
v4l2-ctl --list-formats-ext -d /dev/video0
```

Verify MJPEG 1280x720 30fps exists.

Install Docker

Install Docker:

```
curl -fsSL https://get.docker.com | sh
```

Add user to Docker group:

```
sudo usermod -aG docker $USER
```

Reboot:

```
sudo reboot
```

Verify Docker

```
docker version
```

```
docker compose version
```

Test:

```
docker run hello-world
```

Enable Docker At Boot

```
sudo systemctl enable docker  
sudo systemctl start docker
```

Create Node Directory

```
mkdir -p ~/video-node  
cd ~/video-node
```

Create docker-compose.yml

Create:

```
docker-compose.yml
```

Contents:

```
services:  
  video-node:  
    build: .  
    container_name: video-node  
    restart: unless-stopped  
    network_mode: host  
    privileged: true  
  
  devices:  
    - /dev/video0:/dev/video0  
  
  volumes:  
    - /dev/shm/video-node:/dev/shm/video-node
```

```
- ./config.yaml:/app/config.yaml:ro

environment:
- PYTHONUNBUFFERED=1
- NODE_ID=node-01
```

IMPORTANT:

Change NODE_ID for each node:

```
node-01
node-02
node-03
```

Create Dockerfile

Create:

```
Dockerfile
```

Contents:

```
FROM python:3.12-slim

RUN apt-get update && apt-get install -y
    ffmpeg
    v4l-utils
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

CMD ["python", "main.py"]
```

Create requirements.txt

Create:

```
requirements.txt
```

Contents:

```
fastapi
uvicorn[standard]
pyyaml
aiohttp
python-multipart
```

Create config.yaml

Create:

```
config.yaml
```

Contents:

```
camera:
  device: /dev/video0
  width: 1280
  height: 720
  fps: 30
  input_format: mjpeg

buffer:
  base_dir: /dev/shm/video-node
  segment_count: 10
  segment_seconds: 1

trigger:
  pre_seconds: 3
  post_seconds: 3
  port: 8080

server:
  upload_url: http://192.168.60.1/api/upload
  token: your-secret-token

clip:
  max_retry_seconds: 30
```

IMPORTANT:

The token must match the server configuration.

Copy Application

Copy node application:

```
main.py
```

into:

```
~/video-node/main.py
```

Build And Start Node

Go to project directory:

```
cd ~/video-node
```

Build and start:

```
docker compose up -d --build
```

Verify Node Operation

Check containers

```
docker ps
```

Check logs

```
docker compose logs -f
```

Expected:

```
FFmpeg started
```

Verify health endpoint

```
curl http://localhost:8080/health
```

Expected:

```
{  
  "status": "ok"  
}
```

Verify server sees node

From server:

```
curl http://192.168.60.1/api/nodes
```

Expected:

```
node online
```

Verify Triggering

Manual trigger:

```
curl -X POST http://localhost:8080/trigger
```

Verify:

- clip generated
 - upload succeeds
 - clip visible in browser
-

Verify Trigger-All

From browser:

```
http://192.168.60.1/
```

Press:

```
Trigger All Nodes
```

Verify:

- node records clip
 - upload appears
 - thumbnail generated
-

Useful Commands

View logs

```
docker compose logs -f
```

Restart node

```
docker compose restart
```

Stop node

```
docker compose down
```

Rebuild after code changes

```
docker compose up -d --build
```

Verify webcam modes

```
v4l2-ctl --list-formats-ext -d /dev/video0
```

Verify FFmpeg inside container

```
docker compose exec video-node ffmpeg -version
```

Troubleshooting

Camera not found

Check:

```
ls -al /dev/video0
```

Verify:

- webcam connected
 - only one process uses webcam
 - old services disabled.
-

Trigger works but upload fails

Check:

```
docker compose logs -f
```

Verify:

- server reachable
 - token correct
 - WiFi connected.
-

Node connects to wrong WiFi

Edit:

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Use:

```
priority=100
```

or remove old networks.

WiFi unstable

Verify:

```
country=SE
```

exists.

Recommended:

- disable WiFi power save
 - use 5 GHz AP
 - use official PSU.
-

FFmpeg crashes

Usually caused by:

- insufficient PSU
- USB instability
- unsupported camera format.

Recommended camera mode:

```
MJPEG 1280x720 30fps
```

Final Notes

The node is designed to:

- run fully autonomous
- recover automatically
- reconnect automatically

- buffer continuously in RAM
- upload only triggered events

The architecture supports:

- multiple nodes
- grouped trigger events
- synchronized triggering
- browser playback
- standalone LAN operation
- portable deployments